

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ
по лабораторной работе №2
по дисциплине «Построение и анализ алгоритмов»
Тема: Задача Коммивояжёра

Студентка гр. 4345

Савонькина Е.Н.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Цель работы.

Изучить и реализовать алгоритм решения задачи коммивояжёра. Провести исследование его работы на различных размерах входных данных.

Основные теоретические положения.

Задача коммивояжёра состоит в том, чтобы найти самый короткий замкнутый маршрут, проходящий через все заданные города ровно один раз.

Точный метод реализован через динамическое программирование с мемоизацией. Мы храним состояние в виде маски посещённых городов и номера текущего города. Маска — это двоичное число, в котором единица на i -й позиции означает, что i -й город уже посещён. Рекурсивно вычисляется минимальная стоимость завершения маршрута из текущего состояния. Для этого перебираются все ещё не посещённые города, в которые есть дорога, и выбирается минимум из суммы стоимости перехода и рекурсивно посчитанной стоимости оставшегося пути. Результаты сохраняются в словаре, чтобы не считать одно и то же состояние несколько раз. Когда все города посещены, остаётся только вернуться в начальный город, если дорога есть. После нахождения минимальной стоимости оптимальный путь восстанавливается по сохранённым указателям на лучший следующий город для каждого состояния.

Приближённый метод использует жадный алгоритм АЛШ-1, или алгоритм ближайшего соседа. Он работает быстро, но не гарантирует оптимального решения. Алгоритм начинает движение из стартового города, после чего на каждом шаге ищет ближайший непосещённый город, в который есть прямая дорога, переходит в него и добавляет стоимость перехода к общей сумме. Так продолжается до тех пор, пока не будут посещены все города. После этого делается попытка вернуться в начальный город. Если на каком-то шаге не удаётся найти следующий город или невозможно вернуться обратно, алгоритм сообщает, что путь не найден.

Задание.

Напишите программу, решающую задачу коммивояжёра. Нужно найти кратчайший маршрут, который проходит через все заданные города ровно один раз и возвращается в исходный город. Не все города могут быть напрямую связаны друг с другом.

Входные данные:

n — количество городов ($5 \leq n \leq 15$).

Матрица расстояний между городами размером $n \times n$, где $graph[i][j]$ обозначает расстояние от города i до города j . Если $graph[i][j]=0$ (и $i \neq j$), это означает, что прямого пути между городами нет.

Выходные данные:

Минимальная стоимость маршрута, проходящего через все города и возвращающегося в начальный город.

Оптимальный путь в виде последовательности посещаемых городов, начинающейся и заканчивающейся в городе 0.

Если такого пути не существует, вывести "no path".

Вариант 7.

Точный метод: динамическое программирование (не МВиГ), рекурсивная реализация.

Приближённый алгоритм: АЛШ-1.

Требование перед сдачей: прохождение кода в задании 3.1 на Stepik.

Замечание к варианту 7. АЛШ-1 начинать со стартовой вершины.

Реализация алгоритма.

В начале работы предлагается выбрать способ ввода данных: вручную с клавиатуры или чтением из файла. При ручном вводе сначала задаётся количество городов, затем построчно вводится квадратная матрица расстояний. При файловом вводе первая строка файла содержит количество городов, а последующие строки содержат матрицу. Нулевое значение на пересечении строки i и столбца j при несовпадающих индексах означает отсутствие прямого пути между соответствующими городами.

Основные структуры данных включают глобальную константу `INF`, равную бесконечности, для обозначения невозможности построения маршрута, словарь `memo` для хранения уже вычисленных значений состояний динамического программирования и словарь `parent` для сохранения наилучшего следующего города при восстановлении оптимального пути. Ключом в обоих словарях выступает кортеж из маски посещённых городов и номера текущего города.

Функция `mask_to_binary` является вспомогательной и преобразует целочисленную маску в её двоичное представление фиксированной длины для наглядного отображения в отладочной печати.

Функция `tsp` реализует точный метод решения. Она принимает на вход маску уже посещённых городов и номер текущего города. В теле функции сначала проверяется базовый случай рекурсии: равенство маски значению, в котором все n битов установлены в единицу. Это означает, что все города посещены, и остаётся только вернуться в начальный город. Если дорога из текущего города в нулевой существует, возвращается её стоимость, иначе возвращается бесконечность. Далее проверяется, не было ли уже вычислено данное состояние, и если да, то возвращается сохранённый результат. Основной цикл перебирает все города, пропуская уже посещённые и те, в которые нет прямого пути из текущей позиции. Для каждого допустимого города формируется новая маска добавлением бита этого города, рекурсивно вычисляется стоимость завершения маршрута из нового состояния, и если

она конечна, к ней прибавляется стоимость перехода. Из всех вариантов выбирается минимальный, который вместе с номером лучшего следующего города сохраняется в словарях `memo` и `parent` соответственно.

Функция `nearest_neighbor` реализует приближённый алгоритм АЛШ-1. В начале создаётся список для отметки посещённых городов, текущим объявляется нулевой город, он помечается как посещённый и добавляется в путь. В цикле, выполняющемся n минус один раз, ищется ближайший доступный город из текущего. Для этого перебираются все города, и среди ещё не посещённых с ненулевым расстоянием выбирается город с минимальной стоимостью перехода. Если такой город не найден, функция возвращает `None`, сигнализируя о невозможности построить маршрут. В противном случае выбранный город помечается посещённым, добавляется в путь, стоимость перехода прибавляется к общей сумме, и поиск продолжается уже из него. После посещения всех городов проверяется наличие дороги из последнего города в начальный. Если дороги нет, возвращается `None`. При успешном завершении функция возвращает общую стоимость и построенный путь.

После вызова точного метода проверяется, равна ли полученная стоимость бесконечности. Если равна, выводится сообщение об отсутствии пути. Иначе выводится минимальная стоимость, после чего запускается восстановление оптимального маршрута. Восстановление начинается из начального города с маской, содержащей только первый бит. В цикле, пока маска не станет равна маске со всеми единицами, из словаря `parent` извлекается следующий город для текущего состояния, он добавляется в путь, маска обновляется установкой соответствующего бита, и текущая позиция смещается в этот город. В конце к пути добавляется нулевой город для замыкания цикла. Полученная последовательность городов выводится на экран.

Далее вызывается функция приближённого алгоритма. Если она вернула None, выводится сообщение об отсутствии пути. В противном случае выводятся приближённая стоимость и путь, найденный жадным методом.

Оценка сложности.

Точный метод на основе динамического программирования.

Количество возможных состояний определяется числом различных масок и количеством городов, в которых можно оказаться при каждой маске. Всего существует два в степени n различных подмножеств городов. Для каждого подмножества текущим городом может быть любой из n городов, входящих в это подмножество. Таким образом, общее количество состояний составляет порядка n умножить на два в степени n . Для каждого состояния в худшем случае перебираются все n городов, поэтому итоговая временная сложность алгоритма равна $O(n^2 2^n)$. Объём используемой памяти также пропорционален количеству состояний, то есть $O(n^2 2^n)$, поскольку для каждого состояния в словарях memo и parent хранится ровно одна запись.

Приближённый алгоритм АЛШ-1 имеет полиномиальную временную сложность. На каждом шаге внешнего цикла, который выполняется ровно n минус один раз, происходит полный перебор всех n городов для поиска ближайшего непосещённого соседа. Таким образом, временная сложность составляет $O(n)$. Используемая память минимальна и состоит из списка посещённых городов размера n и списка пути также размера n , то есть объём памяти равен O от n .

Тестирование.

Входные данные	Результат	Комментарий
6 0 5 12 45 34 31 3 0 5 7 9 33 42 46 0 3 31 6 31 12 19 0 35 39 31 12 14 21 0 24 27 21 22 42 48 0	Итоговая стоимость = 99 Итоговый путь: 0 -1 -2 -3 -4 -5 -0 Результат алш-1 Приближённая стоимость = 99 Приближённый путь: 0 1 2 3 4 5 0	Результат корректен.

5 0 10 32 36 42 10 0 9 19 9 32 9 0 49 30 36 19 49 0 16 42 9 30 16 0	Итоговая стоимость = 101 Итоговый путь: 0 -1 -2 -4 -3 -0 Результат алш-1 Приближённая стоимость = 101 Приближённый путь: 0 1 2 4 3 0	Результат корректен.
4 0 34 50 15 20 0 20 40 11 0 0 39 41 22 1 0	Проверяем город 3 Город 3 уже посещён Не удалось найти следующий город Маршрут невозможен Результат алш-1 по path	Результат корректен.
1 0	Стартуем из города 0 Все города посещены Из города 0 нельзя вернуться в 0 Цикл невозможен Результат алш-1 по path	Результат корректен.

Вывод.

В ходе выполнения работы были реализованы два алгоритма для решения задачи коммивояжёра: точный метод на основе динамического программирования с мемоизацией и приближённый алгоритм АЛШ-1, или алгоритм ближайшего соседа.

ПРИЛОЖЕНИЕ А.

ИСХОДНЫЙ КОД ПРОГРАММЫ.

```
INF = float('inf')

print("Выберите способ ввода матрицы:")
print("1 - Ввести вручную")
print("2 - Считать из файла")

choice = input("Ваш выбор: ")

if choice == "1":
    n = int(input("Введите количество городов: "))
    print("Введите матрицу расстояний:")
    graph = [list(map(int, input().split())) for i in range(n)]

elif choice == "2":
    filename = input("Введите имя файла: ")
    with open(filename, "r") as file:
        lines = file.readlines()
    n = int(lines[0])
    graph = []
    for i in range(1, n + 1):
        row = list(map(int, lines[i].split()))
        graph.append(row)
else:
    print("Неверный выбор")
    exit()
memo = {}
parent = {}

def mask_to_binary(mask):
    return format(mask, f'0{n}b')

def tsp(mask, pos):

    print(f"Вызов tsp(mask={mask_to_binary(mask)}, pos={pos})")

    if mask == (1 << n) - 1:

        print("Все города посещены")

        if graph[pos][0] == 0:
            print(f"Из города {pos} нельзя вернуться в 0")
            return INF

        print(f"Возвращаемся из {pos} в 0")
        print(f"Стоимость возврата = {graph[pos][0]}")

        return graph[pos][0]

    if (mask, pos) in memo:

        print("Состояние уже вычислено")
        print(f"memo[{mask_to_binary(mask)}, {pos}] = {memo[(mask, pos)]}")

        return memo[(mask, pos)]

    best = INF
```



```

best_city = -1

print(f"Текущий город: {pos}")
print(f"Текущая маска: {mask_to_binary(mask)}")

for city in range(n):

    print(f"\nПопробуем перейти в город {city}")

    if mask & (1 << city):
        print(f"Город {city} уже посещён")
        continue

    if graph[pos][city] == 0:
        print(f"Дороги из {pos} в {city} нет")
        continue

    print(f"Дорога существует: {pos} - {city}")
    print(f"Стоимость дороги = {graph[pos][city]}")

    new_mask = mask | (1 << city)

    print(f"Новая маска: {mask_to_binary(new_mask)}")

    recursive_cost = tsp(new_mask, city)

    if recursive_cost == INF:
        print(f"Путь через город {city} невозможен")
        continue

    total_cost = graph[pos][city] + recursive_cost

    print(f"Общая стоимость через {city} = "
          f"{graph[pos][city]} + {recursive_cost} = {total_cost}")

    if total_cost < best:

        print(f"Найден новый лучший маршрут через {city}")

        best = total_cost
        best_city = city

memo[(mask, pos)] = best
parent[(mask, pos)] = best_city

print("\nИтог состояния")
print(f"mask = {mask_to_binary(mask)}")
print(f"pos = {pos}")
print(f"best = {best}")
print(f"best_city = {best_city}")

return best

def nearest_neighbor(graph, n):

    print("АЛМ-1")

    visited = [False] * n

    path = [0]
    visited[0] = True

    current = 0

```

```

total_cost = 0

print(f"\nСтартуем из города 0")

for step in range(n - 1):

    print(f"Шаг {step + 1}")

    best_city = -1
    best_dist = INF

    print(f"Текущий город: {current}")

    for city in range(n):

        print(f"\nПроверяем город {city}")

        if visited[city]:
            print(f"Город {city} уже посещён")
            continue

        if graph[current][city] == 0:
            print(f"Нет дороги из {current} в {city}")
            continue

        print(f"Есть дорога {current} - {city}")
        print(f"Стоимость = {graph[current][city]}")

        if graph[current][city] < best_dist:

            print(f"Город {city} - новый лучший сосед")

            best_dist = graph[current][city]
            best_city = city

    if best_city == -1:

        print("Не удалось найти следующий город")
        print("Маршрут невозможен")

        return None, None

    print(f"\nВыбираем город {best_city}")
    print(f"Стоимость перехода = {best_dist}")

    visited[best_city] = True

    path.append(best_city)

    total_cost += best_dist

    print(f"Текущая стоимость маршрута = {total_cost}")
    print(f"Текущий путь: {' - '.join(map(str, path))}")

    current = best_city

print("Все города посещены")

if graph[current][0] == 0:

    print(f"Из города {current} нельзя вернуться в 0")
    print("Цикл невозможен")

    return None, None

```

```

        print(f"Возвращаемся из {current} в 0")
        print(f"Стоимость возврата = {graph[current][0]}")

        total_cost += graph[current][0]

        path.append(0)

        print(f"\nИтоговая стоимость = {total_cost}")
        print(f"Итоговый путь: {' -'}.join(map(str, path))")

        return total_cost, path

print("Точный метод")

exact_cost = tsp(1, 0)

print("Ответ точного метода")

if exact_cost == INF:

    print("no path")

else:

    print(f"Минимальная стоимость = {exact_cost}")

    exact_path = [0]

    mask = 1
    pos = 0

    print("\nВосстановление пути:")

    while mask != (1 << n) - 1:

        next_city = parent[(mask, pos)]

        print(f"{pos} - {next_city}")

        exact_path.append(next_city)

        mask = mask | (1 << next_city)
        pos = next_city

    exact_path.append(0)

    print(f"{pos} - 0")

    print("\nОптимальный путь:")
    print(*exact_path)

approx_cost, approx_path = nearest_neighbor(graph, n)

print("Результат алг-1")

if approx_cost is None:

    print("no path")

else:

    print(f"Приближённая стоимость = {approx_cost}")

```

```
print("Приближённый путь:")  
print(*approx_path)
```